

Lab 03

OpenGL Basic Building Blocks: Points & Lines

Objectives

Other than polygons, OpenGL primitives also include **points** and **lines**. This lab will explore how to use these primitives.

Special Instructions

READ these lab instructions BEFORE starting!

1. In Lab01, you were given 4 files – CGLabmain.cpp, CGLabmain.hpp, CGLab01.cpp and CGLab01.hpp files. We still need the first two files in this lab, but we will create the other NEW files on top of that later.
2. DO NOT copy-and-paste blindly! Some portions of code provided are just modifications, or additions of some lines of code. Always check carefully WHAT and WHERE to copy and paste. Most modifications/additions are in **BOLD**.

Programming Exercises

(90 minutes)

In this lab, we will learn to use points and lines, since lines will be more interesting, we will start with lines first.

Part 1: Using Lines

Task 1A: Draw a Star (15 minutes)

To draw lines, it is very similar to the way we used to draw polygons (from this point on, when we mention polygons, it always include triangles and quads as well, unless explicitly mention otherwise). To draw lines in fact is much easier than drawing polygons.

In this lab, we will create two new files (CGLab03.hpp and CGLab03.cpp) and add them into our project, please refer to Lab02 on how to create and add new files into the project. Also, it is probably good practice to remove files from the earlier labs (Lab02 or Lab01) that are no longer needed in the current project. Although there should not be any conflict if you left them around as they use different namespaces, but remove them for the sake of better project organisation.

Before you do that, just as in Lab02, you will need to modify two lines in CGLabmain.cpp so that these two lines,

```
...  
#include "CGLab01.hpp"  
  
using CGLab01::MyVirtualWorld;  
...
```

will be changed to...

```
...  
#include "CGLab03.hpp"  
  
using CGLab03::MyVirtualWorld;  
...
```

NOTE: Please **REMEMBER** this step. We may not remind you again in future labs.

Since we are going to draw lines, let's draw a star shape using lines.

Just as mentioned above, the new files to be created are CGLab03.hpp and CGLab03.cpp. CGLab03.hpp is given as below:

```
/*  
TCS2221 Computer Graphics  
  
Faculty of IT, Multimedia University  
  
CGLab03.hpp  
  
Objective: Header File for Lab03 on Points & Lines  
  
*/  
  
#ifndef YP_CGLAB03_HPP  
#define YP_CGLAB03_HPP  
  
#include "CGLabmain.hpp"  
  
namespace CGLab03 {  
  
class MyStar  
{  
public:  
    void draw();  
};  
  
//-----  
//the main program will call methods from this class  
class MyVirtualWorld  
{  
public:  
    MyStar mystar;  
  
    MyVirtualWorld()  
    {  
    }  
  
    void draw()  
    {  
        mystar.draw();  
    }  
    void tickTime()  
    {  
        //do nothing, reserved for doing animation  
    }  
  
    //for any one-time only initialization of the  
    //    virtual world before any rendering takes place
```

```
// BUT after OpenGL has been initialized
void init()
{
}
};

}; //end of namespace CGLab03

#endif //YP_CGLAB03_HPP
```

CGLab03.cpp is as below:

```
/*
TCS2221 Computer Graphics

Faculty of IT, Multimedia University

CGLab03.cpp

Objective: Lab03 on Points & Lines
*/

#include <GL/glut.h>
#include "CGLab03.hpp"

#include <cmath>
#include <cstdlib>
#include <ctime>

using namespace CGLab03;

void MyStar::draw()
{
    glLineWidth(3.0); //set the line thickness
    glBegin(GL_LINES);
        glColor3f(1.0, 1.0, 0.0);
        glVertex3f(-10.0, 5.0, 0.0);
        glVertex3f(10.0, 5.0, 0.0);

        glVertex3f(10.0, 5.0, 0.0);
        glVertex3f(-10.0,-10.0, 0.0);

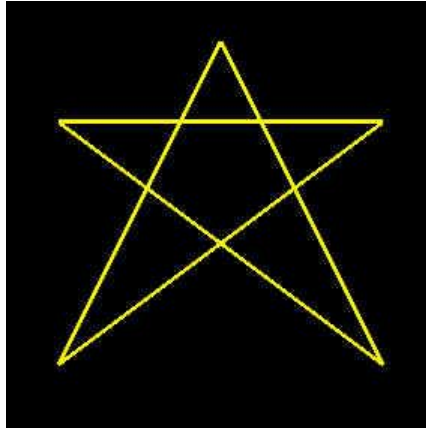
        glVertex3f(-10.0,-10.0, 0.0);
        glVertex3f(0.0, 10.0, 0.0);

        glVertex3f(0.0, 10.0, 0.0);
        glVertex3f(10.0,-10.0, 0.0);

        glVertex3f(10.0,-10.0, 0.0);
        glVertex3f(-10.0, 5.0, 0.0);
    glEnd();
}
```

The codes given above will draw a star shape using yellow lines. Take note that **two vertices will form one single line.**

Now run the program, and you should see the following:



Take note that all the z values of the vertices are zeros, thus the star will appear on the X-Y plane at $z=0$.

Try to play around with the coordinates a bit and make sure you understood the code.

As you may have guessed, the `glLineWidth()` function above is for changing the **line thickness**. try changing the number to a bigger value (let's say 5...) and see the effect. Try to move closer or move away from the shape and you will notice that the thickness of the line remain the same. Why? You need to know that the thickness of line is measured in **screen dimension** and not the world dimension.

Try changing the color by modifying it as highlighted below:

```
void MyStar::draw()
{
    glLineWidth(3.0); //set the line thickness
    glBegin(GL_LINES);
        glColor3f(1.0, 1.0, 0.0);
        glVertex3f(-10.0, 5.0, 0.0);
        glVertex3f( 10.0, 5.0, 0.0);

        glColor3f(1.0, 0.0, 0.0);
        glVertex3f( 10.0, 5.0, 0.0);
        glVertex3f(-10.0,-10.0, 0.0);

        glColor3f(0.0, 0.0, 1.0);
        glVertex3f(-10.0,-10.0, 0.0);
        glVertex3f( 0.0, 10.0, 0.0);

        glColor3f(0.0, 1.0, 0.0);
        glVertex3f( 0.0, 10.0, 0.0);
        glColor3f(1.0, 0.0, 1.0);
        glVertex3f( 10.0,-10.0, 0.0);

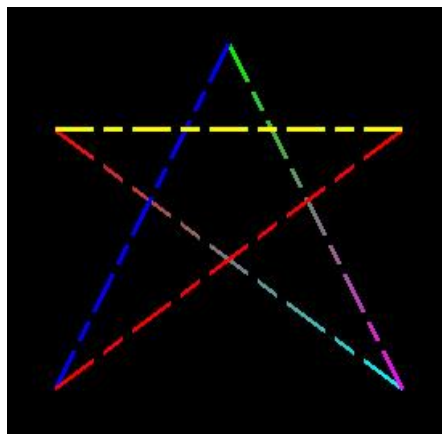
        glColor3f(0.0, 1.0, 1.0);
        glVertex3f( 10.0,-10.0, 0.0);
        glColor3f(1.0, 0.0, 0.0);
        glVertex3f(-10.0, 5.0, 0.0);
    glEnd();
}
```

Now let's add two more lines into our program as shown below:

```
void MyStar::draw()
{
    glEnable(GL_LINE_STIPPLE);
    glLineStipple(3, 0x3cff);
    //3 is a stipple multiplier factor
    //0x3cff (is 00111100 11111111 in binary) is stipple pattern

    glLineWidth(3.0); //set the line thickness
    glBegin(GL_LINES);
        ...
        ...
    glEnd();
    glDisable(GL_LINE_STIPPLE);
}
```

If you run the program, you will now see the effect of the line stipple pattern as shown below:



What do you notice about the relationship between the line stipple pattern as shown above and the input to the `glLineStipple()` function? The best way to understand the line stipple pattern is to experiment with different values. Try one of these:

```
glLineStipple(1, 0x3cff);
glLineStipple(6, 0x3cff);

glLineStipple(1, 0xffff); //same as default
glLineStipple(2, 0xb38f);
```

So, any idea yet? :)

Task 1B: Draw Axes (5 minutes)

Okay, now let's create something very useful. Many people have problems visualizing the 3D space, thus here we will create the axes and a bounding box around our objects.

But wait a minute...we already seen the axes that we can turn on and off using the F2 key as explained in Lab01. Actually, the code is already in the `CGLabmain.hpp` program, so you **DO NOT NEED TO TYPE THIS AGAIN**. Here's part of the code which draws the axes using lines:

```
class MyAxis
{
private:
    GLfloat xlen, ylen, zlen;
    GLfloat xred, xgreen, xblue;
    GLfloat yred, ygreen, yblue;
    GLfloat zred, zgreen, zblue;
    GLint   linestipplerepeatfactor;
    GLfloat linewidth;
    GLushort linestipplepattern;
    ...
public:
    MyAxis()
    {
        setLineStipple(1, 0xfffff);
        setLineWidth(1);
        ...
    }

    void setLineWidth(GLfloat lineWidth)
    { linewidth = lineWidth; }
    void setLineStipple(GLint repeatfactor, GLushort pattern)
    { linestipplerepeatfactor = repeatfactor, linestipplepattern = pattern; }

    void draw()
    {
        ...
        glEnable(GL_LINE_STIPPLE);
        glLineWidth(linewidth);
        glLineStipple(linestipplerepeatfactor, linestipplepattern);
        glBegin(GL_LINES);
            //x-axis
            glColor3f(xred, xgreen, xblue);

            glVertex3f( 0.0, 0.0, 0.0);
            glVertex3f(xlen, 0.0, 0.0);
            //x-axis arrow
            glVertex3f( xlen, 0.0, 0.0);
            glVertex3f(0.9*xlen, 0.0, 0.05*xlen);
            glVertex3f( xlen, 0.0, 0.0);
            glVertex3f(0.9*xlen, 0.0, -0.05*xlen);

            //y-axis
            glColor3f(yred, ygreen, yblue);
            glVertex3f(0.0, 0.0, 0.0);
            glVertex3f(0.0, ylen, 0.0);
            //y-axis arrow
            glVertex3f( 0.0, ylen, 0.0);
            glVertex3f( 0.05*ylen, 0.9*ylen, 0.0);
```

```

        glVertex3f(0.0, ylen, 0.0);
        glVertex3f(-0.05*ylen, 0.9*ylen, 0.0);

        //z-axis
        glColor3f(zred, zgreen, zblue);
        glVertex3f(0.0, 0.0, 0.0);
        glVertex3f(0.0, 0.0, zlen);
        //z-axis arrow
        glVertex3f(0.0, 0.0, zlen);
        glVertex3f(0.05*zlen, 0.0, 0.9*zlen);
        glVertex3f(0.0, 0.0, zlen);
        glVertex3f(-0.05*zlen, 0.0, 0.9*zlen);
    glEnd();
    ...
} //void draw()
...
};

```

Please refer to CGLabmain.hpp for the complete code. Try to understand this as just another example for drawing lines.

Here's a side note about good C++ program design: It is actually NOT a good practice to put a function definition (i.e. the implementation) in the .hpp header file. It should be placed in the .cpp program file instead. The reason we are putting this draw() function of the MyAxis class in CGLabmain.hpp instead of CGLabmain.cpp (or another .cpp file) is to hide it from you in order not to confuse you in previous labs.

Task 1C: Draw Bounding Box (15 minutes)

Another useful thing is a bounding box, which allows us to visualize an enclosed area of space. CGLab03.hpp and CGLab03.cpp of Task 1A are to be further modified.

REMINDER: Please make sure you understood the steps on how to add more classes into the project. Most of the time,

- the **class declaration** should go to the .hpp header file, and
- the **class definition/implementation** should go to the .cpp file.

Add the code in **bold** below into CGLab03.hpp:

```

...
...
class MyBoundingBox
{
public:
    MyBoundingBox( GLfloat xcenter, GLfloat ycenter, GLfloat zcenter,
                  GLfloat xdim, GLfloat ydim, GLfloat zdim )
    {
        GLfloat xdimhalf = xdim/2.0;
        GLfloat ydimhalf = ydim/2.0;
        GLfloat zdimhalf = zdim/2.0;

        y[0] = ycenter + ydimhalf;
        y[1] = y[2] = y[3] = y[0]; //top
        y[4] = ycenter - ydimhalf;
    }
};

```

```

        y[7] = y[6] = y[5] = y[4]; //bottom

        x[1] = xcenter + xdimhalf;
        x[6] = x[5] = x[2] = x[1]; //right
        x[0] = xcenter - xdimhalf;
        x[7] = x[4] = x[3] = x[0]; //left

        z[0] = zcenter + zdimhalf;
        z[5] = z[4] = z[1] = z[0]; //front
        z[2] = zcenter - zdimhalf;
        z[7] = z[6] = z[3] = z[2]; //back
    }

    void draw();
private:
    GLfloat x[8], y[8], z[8]; //(x,y,z) of 8 corner points
};

class MyVirtualWorld
{
public:
    MyStar mystar;
    MyBoundingBox myboundingbox;

    MyVirtualWorld() : myboundingbox(0.0, 0.0, 0.0, 40.0, 40.0, 40.0)
    {
    }

    void draw()
    {
        mystar.draw();
        myboundingbox.draw();
    }
    ...
};
...

```

Add the following code below into CGLab03.cpp:

```

...
void MyBoundingBox::draw()
{
    glLineWidth(1.0);
    glBegin(GL_LINES);
        glColor3f(1.0, 1.0, 0.0);
        glVertex3f(x[0], y[0], z[0]);
        glVertex3f(x[1], y[1], z[1]);
        glVertex3f(x[1], y[1], z[1]);
        glVertex3f(x[2], y[2], z[2]);
        glVertex3f(x[2], y[2], z[2]);
        glVertex3f(x[3], y[3], z[3]);
        glVertex3f(x[3], y[3], z[3]);
        glVertex3f(x[0], y[0], z[0]);

        glVertex3f(x[4], y[4], z[4]);
        glVertex3f(x[5], y[5], z[5]);
        glVertex3f(x[5], y[5], z[5]);
        glVertex3f(x[6], y[6], z[6]);
    }
}

```

```

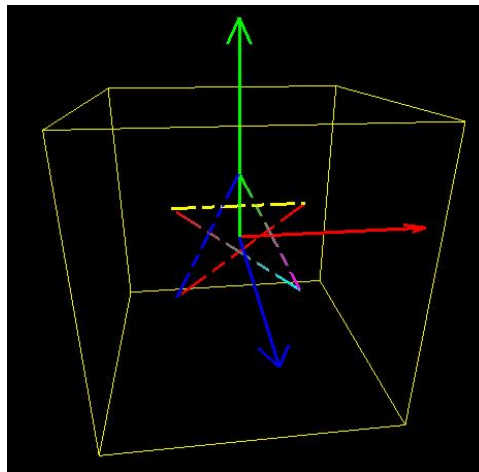
    glVertex3f(x[6], y[6], z[6] );
    glVertex3f(x[7], y[7], z[7] );
    glVertex3f(x[7], y[7], z[7] );
    glVertex3f(x[4], y[4], z[4] );

    glVertex3f(x[0], y[0], z[0] );
    glVertex3f(x[4], y[4], z[4] );
    glVertex3f(x[1], y[1], z[1] );
    glVertex3f(x[5], y[5], z[5] );
    glVertex3f(x[2], y[2], z[2] );
    glVertex3f(x[6], y[6], z[6] );
    glVertex3f(x[3], y[3], z[3] );
    glVertex3f(x[7], y[7], z[7] );
    glEnd();
}
...

```

In `MyBoundingBox`, take notice that we pre-calculate the 8 corners points based on parameters provided. Since `myDisplayFunc()` function (which will call the `void MyBoundingBox::draw()` function) will be called each time there is a need to refresh the screen, thus it would be much more efficient to pre-calculate whatever value in advance rather than to calculate it at every refresh.

If you run the program, you will see the below (with axes turned on):

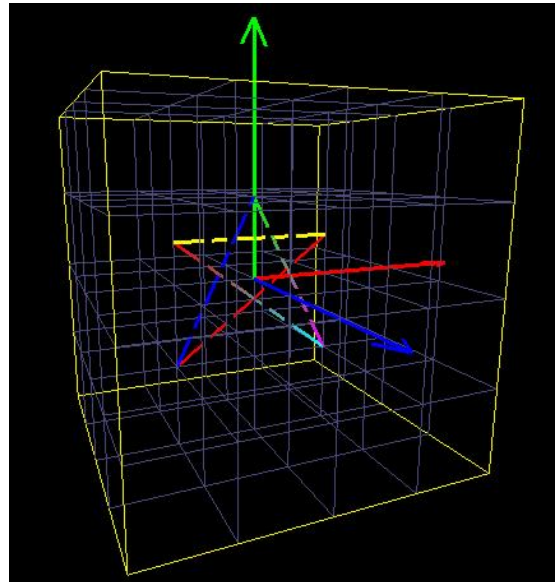


If you wish, you may reduce the line thickness of the box so that the objects (e.g. star) inside the bounding box appear to stand out clearer.

Task 1D: Drawing Grid (25 minutes)

Your next task is to add grid lines into the world, the output should be as given on the right:

Hey, wouldn't there be too many lines to draw?!?
Well, since there are so many lines, you may need to use some **LOOPS** to draw them.

**Part 2: Using Points****Task 2A: Drawing Points** (10 minutes)

To render points, we use `GL_POINTS` on `glBegin(...)`. Each line of `glVertex3f(..., ..., ...)`; represents a single point. You can have as many points as you wish, and you can stop by indicating `glEnd()`.

```
glBegin(GL_POINTS);

    glVertex3f(..., ..., ...);
    glVertex3f(..., ..., ...);
    ...
    ...
    glVertex3f(..., ..., ...);

glEnd();
```

Add the code in **bold** into `CGLab03.hpp` as shown below:

```
...
class MyPoints
{
public:
    void draw();
};

class MyVirtualWorld
{
public:
    MyStar mystar;
    MyBoundingBox myboundingbox;
    MyPoints mypoints;

    MyVirtualWorld()
        : myboundingbox(0.0, 0.0, 0.0, 40.0, 40.0, 40.0)
    {
```

```
    }

    void draw()
    {
        mystar.draw();
        myboundingbox.draw();
        mypoints.draw();
    }
    ...
};
...
```

Add the following into CGLab03.cpp:

```
...
void MyPoints::draw()
{
    glPointSize(1.0);

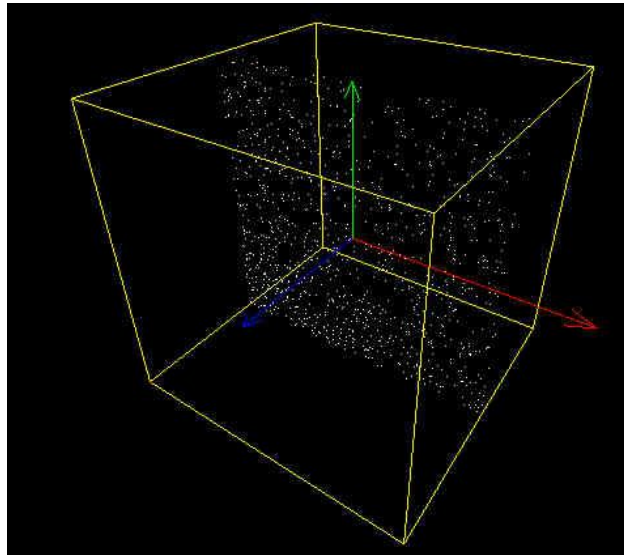
    srand(1); /* fixed random seed, thus
               // the point are fixed
               /* comment this out to get
               // 'force field' effect

    glColor3f(1.0f, 1.0f, 1.0f);

    GLfloat x,y,z;
    glBegin(GL_POINTS);
        for (int i=0; i<1000; ++i)
        {
            x = (rand()%10000)/10000.0 * 40.0 - 20.0;
            y = (rand()%10000)/10000.0 * 40.0 - 20.0;
            z = 0;
            //try this:
            //z = (rand()%10000)/10000.0 * 40.0 - 20.0;
            glVertex3f(x,y,z);
        }
    glEnd();
}
...

```

When you run the program, you will see the below:



Those points are generated using pseudo-random numbers.

Try comment out the `srand(1);` statement and observe what happens...

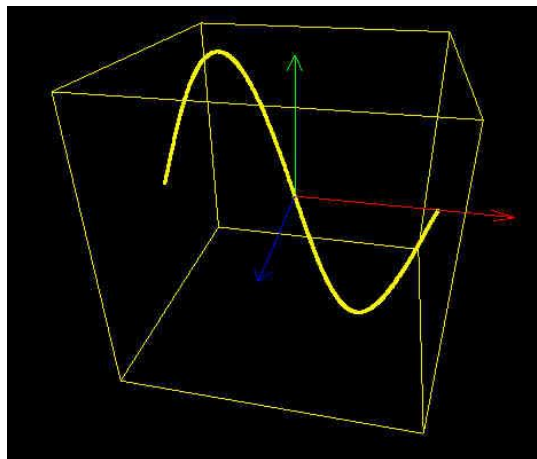
At each refresh, `rand()` takes a new random seed, which in turn generates different random points each time. So, what you observe is a nice “force field” effect!

The `glPointSize()` function above is for changing the size of the points, try changing the number to a bigger value and see the effect. Try to move closer or move away from the points and you will notice that the size of points remain the same. Just as in line thickness, the point size is measured in **screen dimension** and not the world dimension.

Also, try to generate random points along the z-axis as well to obtain a nice 3D outer space effect!

Task 2B: Drawing curvatures using points (20 minutes + Homework)

By just using points, try to draw a sine graph on the X-Y plane, as shown below:



* END OF LAB *